

03_m5_m6b

M5 - Semantic Layer live

Objective

Make the metric registry the single source of truth and put a hard compile gate in front of it. The registry (`models/semantic/semantic_layer.yaml` , registry v2.0.0 - 47 metrics / 19 dimensions / 7 entities / 4 time grains) ALREADY EXISTS and is referential-integrity validated; you do NOT regenerate it. You add (1) a thin dbt schema wrapper so the layer is part of the dbt DAG and documented, and (2) a standalone Python validator that proves every `numerator / denominator` , every derived `expr {token}` , every `dimensions_supported` entry, and every `agents` entry resolves against the live registry. The validator is the M5 exit gate and the CI compile gate referenced by `METRIC_GOVERNANCE_GUIDE.md §6.1` (invariant I1: no metric exists outside the registry).

Files to create

Path	Source to copy from
<code>models/semantic/semantic_layer.yaml</code>	ALREADY EXISTS - do NOT regenerate. This is the keystone (47 metrics).
<code>models/semantic/metrics__schema.yml</code>	New thin wrapper - template below. Spec: <code>METRIC_GOVERNANCE_GUIDE.md §2</code> , <code>CLAUDE.md §5</code> .
<code>scripts/validate_semantic.py</code>	New - FULL code below (the reader has no AI; copy it verbatim). Spec: <code>MCP_ARCHITECTURE.md §7-§8</code> , <code>METRIC_GOVERNANCE_GUIDE.md §6.1</code> .

`metrics__schema.yml` is a documentation/exposure shell - the semantic layer materializes as a `view` (per `CLAUDE.md §5`) and the YAML registry is the governance object. Create it as:

```
# models/semantic/metrics__schema.yml
version: 2
exposures:
  - name: semantic_layer
    type: application
    maturity: high
    owner: {name: analytics-eng, email: analytics-eng@helios}
    description: >
      Helios semantic registry (semantic_layer.yaml, v2.0.0). The single source of
      truth for all 47 metrics / 19 dimensions. semantic-mcp.build_query reads it;
      every governed SQL string derives from it. Referential integrity is enforced
      by scripts/validate_semantic.py (CI compile gate).
    depends_on:
      - ref('fct_funnel')
      - ref('fct_sessions')
      - ref('fct_orders')
      - ref('fct_order_items')
      - ref('fct_cohorts')
```

Now `scripts/validate_semantic.py` - copy verbatim:

```
#!/usr/bin/env python3
"""Referential-integrity compile gate for the Helios semantic registry.

Loads models/semantic/semantic_layer.yaml and asserts, with ZERO dangling refs:
1. every metric's numerator/denominator (ratio) resolves to a defined metric_name
2. every derived metric's expr {token} resolves to a defined metric_name
3. every dimensions_supported entry resolves to a defined dimension_name
4. every agents[] entry is one of the 6 valid metric-consuming agents
5. every metric's grain is a defined grains: key
6. the type → population matrix holds (ratio has num+den; derived has expr)
```

```

7. all canonically-required metric names are present
Exits 1 (and prints FAIL with every error) if any check fails; prints PASS otherwise.
No AI, no network, no warehouse - pure file validation.
"""
import re
import sys
from pathlib import Path

try:
    import yaml
except ImportError:
    sys.exit("FAIL: pyyaml not installed. Run: pip install pyyaml")

REGISTRY = Path(__file__).resolve().parents[1] / "models" / "semantic" / "semantic_layer.yaml"

# The 6 metric-consuming agents (Orchestrator plans but consumes no metrics).
VALID_AGENTS = {"Monitor", "Decompose", "Diagnose", "Prescribe", "Narrator", "Critic"}

# A floor of metric names that MUST exist (headline + funnel + decomposition spine).
REQUIRED_METRICS = {
    "sessions", "users", "new_users", "returning_users", "revenue", "orders",
    "gross_revenue", "net_revenue", "aov", "revenue_per_session", "revenue_per_user",
    "view_item_sessions", "add_to_cart_sessions", "begin_checkout_sessions",
    "purchasing_sessions", "session_conversion_rate", "user_conversion_rate",
    "view_to_cart_rate", "cart_to_checkout_rate", "checkout_to_purchase_rate",
    "traffic_share", "channel_revenue", "channel_conversion_rate",
}

EXPR_TOKEN = re.compile(r"\{([a-zA-Z0-9_]+\})\}") # matches {revenue}, {sessions}, ...

def load_registry(path: Path) -> dict:
    if not path.exists():
        sys.exit(f"FAIL: registry not found at {path}")
    with path.open("r", encoding="utf-8") as fh:
        return yaml.safe_load(fh)

def validate(reg: dict) -> list[str]:
    errors: list[str] = []

    metrics = reg.get("metrics") or []
    dims = reg.get("dimensions") or []
    grains = reg.get("grains") or {}

    metric_names = {m["metric_name"] for m in metrics}
    dim_names = {d["dimension_name"] for d in dims}
    grain_keys = set(grains.keys())

    # Sanity: registry shape.
    if not metric_names:
        errors.append("registry has no metrics:")
    if not dim_names:
        errors.append("registry has no dimensions:")

    for m in metrics:
        name = m.get("metric_name", "<unnamed>")
        mtype = m.get("type")

        # (5) grain must resolve
        grain = m.get("grain")
        if grain not in grain_keys:
            errors.append(f"{name}: grain '{grain}' not in grains: {sorted(grain_keys)}")

        # (1) ratio numerator/denominator must resolve to defined metrics
        if mtype == "ratio":
            for slot in ("numerator", "denominator"):
                ref = m.get(slot)
                if ref is None:
                    errors.append(f"{name}: type=ratio but {slot} is null")
                elif ref not in metric_names:
                    errors.append(f"{name}: {slot} '{ref}' is not a defined metric_name")

```

```

        if m.get("expr") is not None:
            errors.append(f"{name}: type=ratio must leave expr null")

    # (2) derived expr {token}s must resolve to defined metrics
    elif mtype == "derived":
        expr = m.get("expr")
        if not expr:
            errors.append(f"{name}: type=derived but expr is null/empty")
        else:
            tokens = EXPR_TOKEN.findall(expr)
            if not tokens:
                errors.append(f"{name}: derived expr '{expr}' has no {{token}}")
            for tok in tokens:
                if tok not in metric_names:
                    errors.append(f"{name}: expr token '{{tok}}' is not a defined metric_name")

    # type → population matrix for count/sum
    elif mtype in ("count", "sum"):
        if not m.get("sql_definition"):
            errors.append(f"{name}: type={mtype} requires sql_definition")
        for slot in ("numerator", "denominator", "expr"):
            if m.get(slot) is not None:
                errors.append(f"{name}: type={mtype} must leave {slot} null")
        else:
            errors.append(f"{name}: unknown type '{mtype}' (expected count|sum|ratio|derived)")

    # (3) dimensions_supported must resolve to defined dimensions
    for d in (m.get("dimensions_supported") or []):
        if d not in dim_names:
            errors.append(f"{name}: dimensions_supported '{d}' is not a defined dimension_name")

    # (4) agents must be valid
    for a in (m.get("agents") or []):
        if a not in VALID_AGENTS:
            errors.append(f"{name}: agent '{a}' not in {sorted(VALID_AGENTS)}")

    # (6) every required canonical metric is present
    for req in sorted(REQUIRED_METRICS):
        if req not in metric_names:
            errors.append(f"required metric '{req}' missing from registry")

    return errors

def main() → int:
    reg = load_registry(REGISTRY)
    metrics = reg.get("metrics") or []
    dims = reg.get("dimensions") or []
    errors = validate(reg)
    if errors:
        print(f"FAIL: {len(errors)} dangling/invalid reference(s) in {REGISTRY.name}:")
        for e in errors:
            print(f"  - {e}")
        return 1
    print(
        f"PASS: {len(metrics)} metrics, {len(dims)} dimensions - "
        f"0 dangling references (numerator/denominator/expr/dimensions/agents/grain all resolve)."
    )
    return 0

if __name__ == "__main__":
    sys.exit(main())

```

Commands to run

```

# from repo root, venv active (POSIX: source .venv/bin/activate | Windows: .venv\Scripts\activate)
pip install pyyaml # if not already from requirements.txt
python scripts/validate_semantic.py # the M5 gate

```

```
dbt parse # registers the exposure; no warehouse needed
dbt docs generate # optional: surfaces the semantic exposure in docs
```

Tests to run

```
# 1. The gate must print PASS and exit 0 on the shipped registry:
python scripts/validate_semantic.py ; echo "exit=0"
# asserts: 47 metrics / 19 dimensions, 0 dangling references.

# 2. Negative test - prove it FAILS loud on a broken ref (work on a COPY):
cp models/semantic/semantic_layer.yaml /tmp/bad.yaml
# hand-edit /tmp/bad.yaml: change one ratio denominator to 'sessionz' (typo)
# point the script at it temporarily, OR set REGISTRY env-style by copying over a scratch repo;
# expected: "FAIL: 1 dangling/invalid reference(s)" + "denominator 'sessionz' is not a defined metric_name", exit 1.
```

Success criteria

- python scripts/validate_semantic.py prints PASS: 47 metrics, 19 dimensions – 0 dangling references and exits 0.
- A deliberately broken numerator/denominator/ {token} /dimension prints FAIL naming the exact offending reference and exits 1 (gate is real, not cosmetic).
- dbt parse succeeds with the semantic_layer exposure depending on the five mart grains.

Common mistakes

- Registry FILENAME drift: the registry is semantic_layer.yaml (v2.0.0, 47 metrics), NOT the retired semantic_models.yaml (v1, 28 metrics). MCP_ARCHITECTURE.md §3 and CLAUDE.md §5 still cite the old name in places - the live file and mcp_servers.yaml MUST both point at semantic_layer.yaml . Fix: standardize on semantic_layer.yaml everywhere.
- Regenerating the registry: it is the keystone and is already RI-validated. Editing it to "fix" a validator error is backwards - fix the validator/typo, never silently rewrite a governed definition.
- Validating expr with a naive split on commas instead of extracting {token} s: SAFE_DIVIDE({revenue},{users}) has two tokens, both must resolve; a substring match misses them. Use the { ... } regex.
- Treating an unknown agent (e.g. Orchestrator in a metric's agents[]) as fine - the Orchestrator plans but consumes no metrics; only the 6 in VALID_AGENTS are legal.
- Skipping the negative test, so you ship a gate that always prints PASS (it must actually exit 1 on a dangling ref, or it is not a gate).
- Forgetting the type->population matrix: a ratio with a stray expr , or a derived with numerator / denominator set, is malformed even if every name resolves.

M6 - Grounding MCP Pair

Objective

Build the two servers that physically enforce grounding: semantic-mcp (the ONLY path to SQL) and warehouse-mcp (the sole BigQuery client, enforcing the dry-run + byte-budget gate). When this milestone is green, an agent can name a metric and get governed SQL that is cost-checked, executed under budget, and reconciled - with zero hand-authored SQL. This is the L1 grounding spine.

Files to create

Path	Source to copy from
helios/mcp/__init__.py	empty package marker
helios/mcp/base.py	MCP_ARCHITECTURE.md §4 (layout), §5 (error taxonomy), §8 (audit wrapper). Server factory + typed errors + _h(sql) hash + config loader.

Path	Source to copy from
helios/mcp/schemas.py	MCP_ARCHITECTURE.md §6 - pydantic/JSON-Schema I/O for every tool.
helios/mcp/semantic.py	MCP_ARCHITECTURE.md §6.2 + §9 resolver skeleton; binding rules §7. Loads semantic_layer.yaml, runs the §8 RI compile at startup, serves get_metric / list_dimensions / build_query.
helios/mcp/warehouse.py	MCP_ARCHITECTURE.md §6.1 + §9 skeleton (copy verbatim). dry_run / run_query / reconcile / list_tables / describe_table.
mcp_servers.yaml	MCP_ARCHITECTURE.md §3 - FINALIZED. Set semantic-mcp.config.registry: ./models/semantic/semantic_layer.yaml (NOT semantic_models.yaml). Byte budget 5368709120; require_dry_run: true.

Error taxonomy for base.py (closed set, JSON-RPC codes from §5) - define these as exception classes that carry code:

```
# helios/mcp/base.py (essentials)
import hashlib
class MCPError(Exception):
    code = -32000
    def __init__(self, message): super().__init__(message); self.message = message
class UnknownMetric(MCPError): code = -32001
class UnknownDimension(MCPError): code = -32002
class DimensionNotPermitted(MCPError): code = -32003
class InvalidFilter(MCPError): code = -32004
class NotDryRunFirst(MCPError): code = -32011
class ByteBudgetExceeded(MCPError): code = -32012

def normalize_hash(sql: str) → str: # I1 dry-run gate key
    return hashlib.sha256(" ".join(sql.split()).lower().encode()).hexdigest()
```

warehouse.py is the load-bearing gate - copy the MCP_ARCHITECTURE.md §9 skeleton verbatim: dry_run records normalize_hash(sql) into _SEEN_DRYRUN; run_query raises NotDryRunFirst unless the hash was seen (I1) and caps maximum_bytes_billed = min(arg, 5_368_709_120) (I2). semantic.py copies the §9 build_query resolver: _measure() emits agg(sql) AS name for count/sum, SAFE_DIVIDE(SUM(num),SUM(den)) AS name for ratio, expand_expr(expr) AS name for derived; it validates every dim against the metric's dimensions_supported (DimensionNotPermitted) and every name against the registry (UnknownMetric/UnknownDimension). Remember the resolver maps authoring fields to short keys (metric_name→name, sql_definition→sql, aggregation_method→agg, dimensions_supported→dimensions) per METRIC_GOVERNANCE_GUIDE.md §3.

Commands to run

```
pip install mcp google-cloud-bigquery pydantic pyyaml
# auth for warehouse-mcp (least-privilege SA, ADC):
gcloud auth application-default login
export GOOGLE_APPLICATION_CREDENTIALS=/path/to/helios-wh-key.json # Windows: $env:GOOGLE_APPLICATION_CREDENTIALS=...
export HELIOS_WH_TOKEN=<bearer>

# RI compile gate runs at semantic-mcp startup; smoke each server:
python -m helios.mcp.semantic # stdio - should load registry, RI-compile, then idle on stdin
python -m helios.mcp.warehouse # streamable-http on :8443
```

Tests to run

```
pytest helios/mcp/tests -k "semantic or warehouse" -v
```

- semantic-mcp (a) unknown metric/dimension -> UnknownMetric / UnknownDimension, NO SQL emitted; (b) build_query('session_conversion_rate', ['device_category'], window='last_28d') produces the session-keyed subquery form (snapshot match to MCP_ARCHITECTURE.md §6.2); (c) a registry with a dangling ref -> server REFUSES to start; (d) a non-whitelisted dim -> DimensionNotPermitted.

- warehouse-mcp guardrail chain: (a) `run_query` without a prior `dry_run -> NotDryRunFirst`; (b) `max_bytes_billed` capped at the 5 GiB budget, over-scan -> `ByteBudgetExceeded`; (c) `reconcile(metric, grain)` vs a hand-written control query within 0.5%; (d) a write SQL -> error (SA is read-only).
- Round-trip: `build_query` → `dry_run` → `run_query` → `reconcile` returns rows and reconciles to ≤0.5%.

Success criteria

- `build_query` → `dry_run` → `run_query` → `reconcile` round-trips and reconciles within 0.5%.
- `run_query` refuses any SQL not dry-run'd this run (`NotDryRunFirst`); byte budget caps at 5 GiB (`ByteBudgetExceeded` on over-scan).
- An unknown metric is a HARD error (`UnknownMetric`), never a fallback to free SQL.
- semantic-mcp refuses to start if the registry has a dangling reference (fail loud).

Common mistakes

- Letting the LLM author raw SQL: the model passes only string names; physical columns live exclusively in registry `sql` fields. The only path to data is `build_query` → `dry_run` → `run_query` . No agent holds a raw-SQL tool.
- Calling `run_query` without a prior `dry_run` , or not capping bytes: both invariants (l1 sha256 gate, l2 byte cap) are preconditions enforced in `warehouse.py` , not agent goodwill. The hash must be normalized identically in `dry_run` and `run_query` (whitespace-collapsed, lowercased) or the gate spuriously fails.
- Registry filename drift in `mcp_servers.yaml` : `semantic-mcp.config.registry` MUST be `./models/semantic/semantic_layer.yaml` . Pointing at the retired `semantic_models.yaml` loads the wrong (28-metric) layer.
- Treating an unknown metric name as a fallback to free SQL instead of a hard error (rule G5). The branch hard-stops and the agent re-plans against `list_dimensions()` /the registry.
- Giving warehouse credentials to the stdio servers: only `warehouse-mcp` (and later `report-mcp` for memory) touch credentials. `semantic/stats/experiment` run credential-free over stdio (least privilege).
- `SELECT *` over `events_*` or scanning all 90+ shards: governed SQL queries the marts (`helios.marts.fct_*`), not raw events; prune with the window predicate, and let `dry_run` reject over-budget scope so the agent narrows dims/window.
- Averaging per-segment ratios: the resolver emits `SAFE_DIVIDE(SUM(num),SUM(den))` AFTER grouping for ratios - never `AVG(per_row_ratio)` (Simpson's-paradox defense).

M6b - Stats / Experiment / Report MCP

Objective

Build the three data-independent servers that complete the L1 toolset: `stats-mcp` (the ONLY path to math - seeded, deterministic), `experiment-mcp` (powered test cards, no data access), and `report-mcp` core (`render_brief` / `export`). The keystone here is `decompose_change` , which dissolves Simpson's paradox by splitting ΔR into mix/rate/interaction - it fails silently if wrong, so it gets a golden-value test FIRST.

Files to create

Path	Source to copy from
<code>helios/mcp/stats.py</code>	MCP_ARCHITECTURE.md §6.3 + §9 <code>decompose_change</code> skeleton (copy verbatim). Seeded <code>rng_seed: 1729</code> . <code>detect_anomaly</code> / <code>decompose_change</code> / <code>significance_test</code> / <code>forecast</code> / <code>cohort_retention</code> / <code>rfm_segment</code> .
<code>helios/mcp/experiment.py</code>	MCP_ARCHITECTURE.md §6.4 . <code>power_analysis</code> / <code>runtime_estimate</code> / <code>design_experiment</code> . Validates target metric against the registry (no data access).
<code>helios/mcp/report.py</code>	MCP_ARCHITECTURE.md §6.5 core only: <code>render_brief</code> / <code>export</code> . (Memory tools <code>save_diagnosis</code> / <code>recall_prior</code> come in M8 - do NOT build them here; report-mcp gets NO BigQuery client.)

Copy the `decompose_change` skeleton from `MCP_ARCHITECTURE.md §9` verbatim - it implements the FOUNDATION identity exactly ($\text{mix} = \sum \Delta w \cdot r_0$, $\text{rate} = \sum w_0 \cdot \Delta r$, $\text{interaction} = \sum \Delta w \cdot \Delta r$, $\Delta R = \text{mix} + \text{rate} + \text{interaction}$) and raises `SegmentMismatch` when `t0 / t1` segment sets differ. `experiment.py`'s `design_experiment` validates `metric` (and guardrail metrics) against the registry so a test card can only target governed metrics. `report.py`'s `render_brief([])` raises `EmptyFindings` (-32040); it has NO warehouse client (least-privilege - rendering is pure string work).

The §6.2 golden example for the `decompose_change` test (two segments, weights shift, rates flat):

```
# helios/mcp/tests/test_decompose_golden.py
from helios.mcp.stats import decompose_change

def test_decompose_change_golden():
    # §6.2 example: weight moves desktop→mobile, both rates UNCHANGED.
    t0 = [{"seg": "desktop", "w": 0.40, "r": 0.030}, {"seg": "mobile", "w": 0.60, "r": 0.012}]
    t1 = [{"seg": "desktop", "w": 0.30, "r": 0.030}, {"seg": "mobile", "w": 0.70, "r": 0.012}]
    out = decompose_change("session_conversion_rate", "device_category", t0, t1)
    assert abs(out["mix_effect"] - (-0.0018)) < 1e-9 #  $\sum \Delta w \cdot r_0 = (-0.10)(0.030) + (0.10)(0.012)$ 
    assert abs(out["rate_effect"] - 0.0) < 1e-9 # rates unchanged
    assert abs(out["interaction"] - 0.0) < 1e-9 # no  $\Delta r$ , so  $\Delta w \cdot \Delta r = 0$ 
    assert abs(out["delta_R"] - (-0.0018)) < 1e-9 # = mix + rate + interaction
```

$\text{mix} = (0.30 - 0.40) \cdot 0.030 + (0.70 - 0.60) \cdot 0.012 = -0.0030 + 0.0012 = -0.0018$; rate and interaction are 0 because no segment rate moved. This is the canonical "the blended rate fell purely because traffic shifted to lower-converting mobile" result - mix-shift, not behavior change.

Commands to run

```
pip install scipy statsmodels prophet pandas mcp pyyaml # pmdarima as a prophet fallback
# smoke each stdio server:
python -m helios.mcp.stats
python -m helios.mcp.experiment
python -m helios.mcp.report
```

Tests to run

```
pytest helios/mcp/tests -v
```

- stats-mcp GOLDEN: `decompose_change` on the §6.2 example -> `mix_effect=-0.0018`, `rate_effect=0`, `interaction=0`, `delta_R=-0.0018` ($\pm 1e-9$). Identity `mix+rate+interaction == delta_R` holds on random inputs. `significance_test` matches a scipy reference. Determinism: same seed (1729) -> byte-identical outputs.
- experiment-mcp: `power_analysis(baseline, mde, alpha=0.05, power=0.8)` two-proportion sample size matches the closed-form reference; `design_experiment` REJECTS an ungoverned metric (`UnknownMetric`).
- report-mcp: `render_brief([])` -> `EmptyFindings`; a brief's every number traces to a backing tool-output hash (faithfulness rule).

Success criteria

- `decompose_change` golden test passes exactly: `mix=-0.0018`, `rate=0`, `interaction=0` on the §6.2 example.
- `power_analysis` matches the closed-form two-proportion sample size.
- `render_brief([])` raises `EmptyFindings` (and the loop emits a "no material change" brief).
- `design_experiment` cannot target a metric absent from the registry.
- Re-running any stats tool on the same input is byte-identical (seed 1729).

Common mistakes

- Computing statistics in prose instead of via stats-mcp: anomaly scores, decompositions, significance, power, forecasts are tool outputs verbatim (rule G2). The LLM never does arithmetic; numbers in the brief come from `stats / experiment` outputs.
- Getting `decompose_change` subtly wrong - it fails SILENTLY (wrong numbers, not an error). Use the §9 formulas exactly: `mix` uses `r(t0)`, `rate` uses `w(t0)`; swapping the `t0 / t1` baselines flips the attribution. The golden test is the only thing that

catches this.

- Not seeding the RNG (`rng_seed: 1729`): non-determinism breaks the byte-identical test and makes eval grading impossible. Seed `scipy/statsmodels/prophet` sampling at import.
- Giving report-mcp a BigQuery client: least-privilege violation. report-mcp core renders strings only; memory I/O (with the `helios_memory` write scope) is a separate M8 concern.
- Skipping the `render_brief([])` -> `EmptyFindings` path: a clean run must still produce a "no material change" brief, not crash or fabricate a finding.
- `design_experiment` not validating its target/guardrail metrics against the registry: a test card could otherwise target a non-existent metric (e.g. `refund_rate`), reintroducing ungoverned names. Validate against `semantic_layer.yaml` .
- Treating `SegmentMismatch` as ignorable: if `t0` / `t1` segment sets differ, the decomposition is undefined - align the segment sets (or the mix term double-counts appearing/disappearing segments).